

Критерии для оценки тестов

Гоголев Иван

5 марта 2026 г.

Содержание

1	Введение	1
2	Экспертная оценка	2
3	Критерии неиспользующие семантику	2
3.1	Минимальный/максимальный размер теста	2
3.2	Размах	2
3.3	Равномерность	2
3.4	Пример расчета	3
3.5	Программа	3
4	Критерии использующие семантику	3
4.1	Граничность	4
4.2	Минимальность и максимальность	4
4.3	Пример вычисления	4
4.4	Автоматический расчет критериев	4
5	Критерии для архивных задач	5
5.1	Простое сравнение	5
5.2	Избыточность	5
5.3	Время проверки	5
6	Заключение	5

1 Введение

Для оценки различных подходов к генерации тестов, необходимо разработать критерии. Примеры расчетов критериев для различных наборов тестов будем проводить на следующих примерах.

№	Условие	Ограничения
1	Найти сумму двух чисел а и b	$0 \leq a, b \leq 10^9$
2	Для данного простого p найти первообразный корень	$2 \leq p \leq 10^6$, p - простое

В соответствии с классическими подходами к разработке тестов критерии должны отражать следующие свойства:

1. Наличие тестов, находящихся на границе пространства тестов
2. Равномерность заполнения пространства тестов

Назовем критерием функцию $K : T \mapsto M, M \subseteq R$, где T - множество наборов тестов, R - множество действительных чисел. Конкретный тестовый набор будем обозначать буквой t .

2 Экспертная оценка

При ручной подготовке тестов основным критерием качества является оценка автора набора тестов $K_1 : T \mapsto \{0, 1\}$. При этом работа по подготовке тестов продолжается пока $K_1(t) \neq 1$

При оценке таким способом автоматической генерации тестов можно использовать более сложные функции

1. $K_2 : T \mapsto \{0, \dots, n\}$
2. $K_3 : T \mapsto [0, 1]$

3 Критерии неиспользующие семантику

Данные критерии используют для тестов модель черного ящика.

3.1 Минимальный/максимальный размер теста

Пусть тестовый набор записан в памяти компьютера в некоторой кодировке тогда $K_4(t) := \min(S)$, где S - множество размеров тестов в памяти компьютера, аналогично определяется K_5 - максимальный размер теста.

3.2 Размах

Определим размах: $K_6(t) := K_5(t) - K_4(t)$

3.3 Равномерность

Из равномерности распределения кодировок на множестве бинарных строк следует равномерность заполнения тестами пространства тестов. Простейшей проверкой этого является следующее: Пусть $s_1 \leq s_2 \leq \dots \leq s_n$ - размеры тестов в отсортированном порядке. Определим $K_7 = \min(s_{i+1} - s_i), i < n$.

3.4 Пример расчета

Рассмотрим расчет критериев на нескольких наборах для задачи №2:

1. 2, 3, 5, 7
2. 2, 999863
3. 2, 999983, 99991, 99989, 9973, 997, 97, 31, 19, 7

Расчет критериев представлен в таблице, все размеры измерялись в байтах.

№	s_i	K_4	K_5	K_6	K_7
1	1,1,1,1	1	1	0	0
2	1,6	1	6	5	5
3	1,1,2,2,2,3,4,5,5,6	1	6	5	1

3.5 Программа

Для расчета тестов была написана простая программа на python

```
import os
def fun(arr):
    ans = 0
    for i in range(1, len(arr)):
        ans = max(ans, arr[i] - arr[i - 1])
    return ans
s = input()
arr = []
for x in os.listdir(s):
    file_path = s + "/" + x
    arr.append(os.path.getsize(file_path))

arr = sorted(arr)

print(arr)
print("K4", arr[0])
print("K5", arr[-1])
print("K6", arr[-1] - arr[0])
print("K7", fun(arr))
```

4 Критерии использующие семантику

К данным критериям отнесем критерии использующие дополнительные данные о задаче кроме набора тестов.

4.1 Граничность

При написании валидаторов необходимо указывать для каждого параметра из ввода допустимые границы. Обозначим: v_i - i -ая проверка в валидаторе, $test$ - тест из набора t . Определим $B(v_i, test) = -1$, если в валидаторе i параметр принял значение нижней допустимой границы, $B(v_i, test) = 1$, если этот параметр равен верхней границе и $B(v_i, test) = 0$ иначе. Определим $B(test) = \sum |B(v_i, test)|$

Тогда граничность $K_8(t) := \max B(test), test \in t$

4.2 Минимальность и максимальность

Запишем последовательность $B(v_1, test), B(v_2, test), \dots, B(v_k, test)$. Определим минимальность теста $M(test) :=$ длина максимального префикса, состоящий полностью из -1. Определим минимальность набора $K_9(t) := \max M(test), test \in t$. Аналогично определяется максимальность теста и набора K_{10} .

4.3 Пример вычисления

Для задачи № 1 рассмотрим несколько наборов тестов

1. 0 0, 1 1
2. 10^9 10^9
3. 2 3, 5 6, 0 10^9

Пусть имеется следующий валидатор

```
#include <iostream>
#include "testlib.h"
using namespace std;
int main(int argc, char *argv[]) {
    registerValidation(argc, argv);
    inf.readInt(0, 1000000000, "a");
    inf.readSpace();
    inf.readInt(0, 1000000000, "b");
    inf.readEoln();
    inf.readEof();
}
```

Тогда $K_8(t_1) = K_8(t_2) = K_8(t_3) = 2$, $K_9(t_1) = 1$, $K_{10}(t_2) = 1$.

4.4 Автоматический расчет критериев

Для расчета критериев достаточно построить синтаксический анализатор, который преобразует валидатор в программу для проверки условий.

5 Критерии для архивных задач

Так как архивные задачи обладают набором посылок с соответствующими вердиктами для таких задач могут быть разработаны дополнительные критерии.

Пусть $s_1, s_2, \dots, s_m$ - набор решений. Определим $h : T \mapsto \{0, 1\}^m$, $h(t)_i = 1$, если решение проходит набор тестов и $h(t)_i = 0$, если не проходит. Определим частичный порядок на векторах: $a \leq b$, если для всех $1 \leq i \leq m$, $a_i \leq b_i$. h обладает следующим свойством объединения тестов $h(t_1 \cup t_2) \leq h(t_1)$ и $h(t_1 \cup t_2) \leq h(t_2)$

5.1 Простое сравнение

Если $h(t_1) \leq h(t_2)$, то t_1 есть более сильный набор тестов. Если два набора являются несравнимыми то их необходимо объединить в новый набор.

5.2 Избыточность

Пусть $t = \{test_1, test_2, \dots, test_k\}$, где $test_i$ - отдельный тест из набора. Тогда если для какого-нибудь i $h(t \setminus \{test_i\}) = h(t)$, то удаление i -го теста не приведет к ухудшению всего набора.

5.3 Время проверки

Многие участники обращают внимание на быстроту работы проверяющей системы. Поэтому тесты также можно сравнивать по среднему времени проверки на заданном наборе решений

6 Заключение

Были предложены критерии, которые будут использованы для оценки различных подходов к автоматической генерации тестов.